



Análise e Desenvolvimento de Sistemas

**ALGORITMOS E ESTRUTURA DE
DADOS EM PYTHON**

RELATÓRIO DE AULAS PRÁTICAS

Nome: Vitória Freitas RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2025

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO ALGORITMOS EM PYTHON

Roteiro 1

O que é lógica de programação e sua importância

É a capacidade organizar as ideias, para que possamos passar as corretas instruções e comandos, no qual farão o computador realizar uma tarefa, assim solucionando algum problema que temos que resolver. Podemos quebrar o problema em partes pequenas, resolvendo ele de pouco em pouco.

Ela é importante, pois com ela os desenvolvedores podem construir códigos mais legíveis e eficientes.

pseudocódigo e fluxograma

Pseudocódigo é uma linguagem intermediária entre a nossa linguagem natural e a de programação, usada para descrever os algoritmos. O fluxograma é um diagrama usado para descrever visualmente um algoritmo, representando uma sequência de operações.

vantagens de usar Python para aprender programação

Entre as diversas vantagens de se aprender Python, uma delas é seu fácil aprendizado, sendo ótima para iniciantes, por conta de sua sintaxe mais simples de se entender e sua extensa biblioteca, facilitando a construção de algoritmos.

Definição de algoritmo, variável, condicional, laço

Um algoritmo pode ser definido como instruções em sequência, que são bem definidas e finitas, são usados para resolver problemas ou realizar alguma tarefa.

Variável é o nome dado para os valores que vão ser manipulados e utilizados durante a operação de um programa.

Uma estrutura condicional é uma estrutura que se uma determinada condição for atendida o algoritmo vai uma decisão. Um laço, também conhecido como estrutura de repetição, é utilizado para repetir uma tarefa, até que a condição seja atendida.

Código fonte das atividades do roteiro e realizadas em sala de aula:

Exercício 1:

```
vezes = int(input("Quantas vezes você vai digitar: "))

for i in range(1, vezes + 1):
    num = int(input("Digite um numero: "))
    if(num % 2 == 0):
        print(f"{num} é par.")
    else:
        print(f"{num} é impar")
```

Exercício 2:

```
nome = input("Seu nome: ").strip()
# fazendo verificação, não se pode dividir por zero
while True:
    qtd = int(input("Quantas valores você vai digitar: "))
    if qtd != 0:
        break
    else:
        print("Insira um número maior que zero")

soma = 0.0

for i in range(qtd):
    valor = float(input("Valor: "))
    soma = soma + valor

media = soma / qtd

if media >= 70:
    status = "Excelente"
elif 50 <= media <= 69:
    status = "Bom"
else:
    status = "Precisa melhorar"

print("\n ---- resultado ---- ")
print(f"Aluno(a): {nome}")
print(f"Media: {media:.2f}")
print(f"Classificação: {status}")
```

Referências

<https://www.locaweb.com.br/blog/temas/codigo-aberto/logica-de-programacao-o-que-e/>

<https://www.facom.ufu.br/~backes/gci007/Aula02-AlgoritmosFluxogramas.pdf>

<https://king.host/blog/tecnologia/python/>

<https://blog.geekhunter.com.br/o-que-e-algoritmo/>

<https://gaea.com.br/variaveis-programacao/>

<https://www.treinaweb.com.br/blog/estruturas-condicionais-e-de-repeticao>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ESTRUTURAS DE DADOS LINEARES EM PYTHON: LISTAS, PILHAS, FILAS E EFICIÊNCIA. NOTAÇÃO BIG-O

Roteiro 2

Definição de listas, pilhas, filas e introdução à notação Big-O

Listas na programação é uma estrutura de dados que são usadas para armazenar um conjunto de dados em uma única variável.

Pilhas também são uma estrutura de dados, elas guardam o valor de maneira sequencial, como se estivesse empilhando um acima do outro, onde o último a entrar é primeiro a sair (LIFO, Last In First Out).

Filas, por outro lado, seguem o padrão FIFO (First In First Out), no qual o primeiro elemento a ser inserido é primeiro a sair.

A notação Big-O descreve a complexidade de um algoritmo em relação ao tamanho da entrada, utilizando termos algébricos. Por exemplo, $O(n^2)$, com n sendo o tamanho da entrada.

Código fonte das atividades do roteiro e realizadas em sala de aula:

```
from collections import deque

fila = deque()

fila.append("Cliente A")
fila.append("Cliente B")
fila.append("Cliente C")
# utilizando o popleft, pois é mais eficiente que o pop(0)
atendido = fila.popleft()

print("Atendido: ", atendido)
print("fila agora: ", list(fila))

fila.append("Cliente D")
print("Proximo a ser atendido", fila[0])
```

```
# atividade realizada para ver a diferença no tempo de execução do pop(0) e
o popleft()
from collections import deque
```

```

import time
n = 20000

lst = list(range(n))
t0 = time.time()

while lst:
    lst.pop(0)

t1 = time.time()
tempo_list = t1 - t0

dq = deque(range(n))
t0 = time.time()

while dq:
    dq.popleft()

t1 = time.time()
tempo_deque = t1 - t0

print(f"Tempo list.pop(0): {tempo_list:.3f} s")
print(f"Tempo deque.popleft: {tempo_deque:.3f} s")

```

```

from collections import deque

pilha = []
pilha.append("prato 1")
pilha.append("prato 2")
print(pilha.pop())

fila = deque()
fila.append("cliente 1")
fila.append("cliente 2")
print(fila.popleft())

```

```

import time

```

```

fila = deque()
pilha = []
t0 = time.time()
for i in range(3):
    nome = input("Nome do cliente: ")
    fila.append(nome)

print("\n Iniciando atendimentos.")

while fila:
    cliente = fila.popleft()
    print(f"Atendendo {cliente}")

for i in range(3):
    tarefa = input(f"Tarefa {i+1}: ")
    pilha.append(tarefa)

print(f"\nExecutando tarefa {i+1}")

while pilha:
    print(f"Executando: {pilha.pop()}")
t1 = time.time()

tempo_de_execucao = t1 - t0
print("Tempo de execução: ", tempo_de_execucao)

```

Referências

<https://cursos.alura.com.br/forum/topico-explicacao-o-que-sao-listas-317465>
<https://www.treinaweb.com.br/blog/o-que-e-e-como-funciona-a-estrutura-de-dados-pilha>
<https://www.treinaweb.com.br/blog/o-que-e-e-como-funciona-a-estrutura-de-dados-fila>
<https://www.freecodecamp.org/portuguese/news/o-que-e-a-notacao-big-o-complexidade-de-tempo-e-de-espaco/>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ESTRUTURAS DE DADOS NÃO LINEARES - ÁRVORES E GRAFOS EM PYTHON

Roteiro 3

Definição de árvore e grafo, vantagens de cada estrutura

Árvores é uma estrutura de dados que organiza os dados de forma hierarquia. Toda árvore é composta por nós e arestas, as arestas são o que ligam esses nós. As árvores possuem um nó chamado de raiz, que é o nó inicial dela. Todos nós superiores são chamados de nós pais e os inferiores de nós filhos, os que não possuem filhos são chamados de folhas.

Grafo é uma estrutura de dados que representa um conjunto de objetos e a relação entre eles, é constituído por vértices e arestas. As vértices, também chamadas de nós e as arestas, é o que os conecta.

Essas estruturas são essenciais para computação, pois são uma ótima maneira de organizar seus dados para diversos programas. As árvores, por exemplo, podem representar um catálogo de produtos, onde o nó raiz é a categoria (como eletrônicos), e os nós filhos são os produtos que tem dentro dessa categoria. Os grafos podem representar caminhos de uma cidade até outra, ligando as que possuem uma conexão, assim podemos mostrar qual é o caminho mais curto de uma cidade até a outra ou todos caminhos possíveis.

Escolha de Python para ilustrar algoritmos de percursos

Python é uma excelente linguagem para construção desses algoritmos de percursos, tanto para alguém que está iniciando os estudos sobre esses algoritmos, tanto para pessoas experientes, pois com ela temos um conjunto de bibliotecas que facilita a construção desses algoritmos, deixando de forma mais legível e às vezes mais eficiente.

Um exemplo de uma forma mais eficiente de construir um algoritmo utilizando uma biblioteca Python, seria na construção de um algoritmos bfs (Busca em Largura) usando o “popleft” que é um método que vem de uma biblioteca de Python chamada de “deque”, com ela a execução do programa fica mais rápida do que ficaria usando o

“pop(0)”, principalmente, caso a quantidade de dados seja grande.

Códigos-fonte das atividades do roteiro

Atividade 1:

```
# classe para o nó
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.filhos = []

    def add_child(self, *novos_filhos):
        self.filhos.extend(novos_filhos)

def imprimir_arvore(no, nivel=0):
    indent = " " * (4 * nivel)
    print(f"{indent}- {no.valor}")
    for filho in no.filhos:
        imprimir_arvore(filho, nivel + 1)

anakin = Node("Anakin Skywalker")
luke = Node("Luke Skywalker")
leia = Node("Leia Skywalker")
ben = Node("Ben Solo")

leia.add_child(ben)
anakin.add_child(luke, leia)

if __name__ == "__main__":
    imprimir_arvore(anakin)
```

Atividade 2:

```
from collections import deque
def bfs(grafo, origem, destino):
```

```

# fila para o bfs, começa apenas com a origem
fila = deque([[origem]])

# visitados para o bfs, começa apenas com a origem
visitados = {origem}

# enquanto tiver item na fila ele ira buscar ou se encontrar o caminho
mais curto ele retorna o caminho
while fila:
    caminho = fila.popleft()
    atual = caminho[-1]
    print("caminho: ", caminho)
    print("atual: ", atual)

    if atual == destino:
        return caminho

    for vizinho in grafo.get(atual, []):
        if vizinho not in visitados:
            visitados.add(vizinho)
            fila.append(caminho + [vizinho])

    return None

# grafo das cidades
grafo_cidades = {
    "São Paulo": ["Rio de Janeiro", "Curitiba"],
    "Rio de Janeiro": ["São Paulo", "Belo Horizonte"],
    "Curitiba": ["São Paulo", "Florianópolis"],
    "Belo Horizonte": ["Rio de Janeiro", "Brasília"],
    "Florianópolis": ["Curitiba"]
}

#inicialização
if __name__ == "__main__":
    origem = input("Cidade de origem: ").strip()
    destino = input("Cidade de destino: ").strip()

```

```

caminho = bfs(grafo_cidades, origem, destino)

if caminho:
    print(f"O percurso é: {caminho}")
    print(f"O número de etapas é {len(caminho) - 1}.")
else:
    print("Não existe caminho entre as duas cidades nesse grafo.")

```

Desafio:

```

from collections import deque
# função bfs
def bfs(grafo, origem, destino):
    fila = deque([[origem]])
    visitados = {origem}

    while fila:
        caminho = fila.popleft()
        atual = caminho[-1]

        if atual == destino:
            #aqui irá retornar tanto a distancia quanto o caminho
            distancia_total = 0
            for i in range(len(caminho) - 1):
                cidade_atual = caminho[i]
                proxima_cidade = caminho[i + 1]
                distancia_total += grafo[cidade_atual][proxima_cidade]

            return caminho, distancia_total

        for vizinho in grafo.get(atual, []):
            if vizinho not in visitados:
                visitados.add(vizinho)
                fila.append(caminho + [vizinho])

    return None, 0

```

```

#grafo
grafo_cidades = {
    "São Paulo": {
        "Rio de Janeiro": 400,
        "Curitiba": 410
    },
    "Rio de Janeiro": {
        "São Paulo": 400,
        "Belo Horizonte": 440
    },
    "Curitiba": {
        "São Paulo": 410,
        "Florianópolis": 300
    },
    "Belo Horizonte": {
        "Rio de Janeiro": 440,
        "Brasília": 720
    },
    "Florianópolis": {
        "Curitiba": 300
    },
    "Brasília": {
        "Belo Horizonte": 720
    }
}

#inicialização
if __name__ == "__main__":
    origem = input("Cidade de origem: ").strip()
    destino = input("Cidade de destino: ").strip()

    caminho, distancia = bfs(grafo_cidades, origem, destino)

    if caminho:
        print(f"O percurso é: {caminho}")
        print(f"O número de etapas é {len(caminho) - 1}")
        print(f"Distância total: {distancia} km")

```

```
else:  
    print("Não existe caminho entre as duas cidades nesse grafo.")
```

Conclusão

A aula junto com os exercícios propostos no roteiro foram essenciais para o meu entendimento sobre grafos e árvores, com a aula entendemos melhor sua teoria e com os exercícios, conseguimos praticar, assim guardamos a informação ainda mais para um melhor aprendizado.

Referências

<https://wikiciencias.casadasciencias.org/wiki/index.php/Grafo>

<https://algor.dev/arvores-estrutura-de-dados/>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE ORDENAÇÃO EM PYTHON (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort)

Roteiro 4

Definir ordenação e justificar sua relevância em ciência da computação

Ordenação é a ação de deixar os valores em uma ordem, como por exemplo, pegar uma array de números inteiros e ordenar do menor para o maior, isso para computação é importante, pois assim desenvolvedores podem organizar os seus dados, facilitando na hora da busca, em algoritmos como o de busca binária, que possui uma ótima eficiência, precisa que os dados estejam previamente ordenados.

Explicar diferenças conceituais entre algoritmos quadráticos e log-lineares

Nos algoritmos quadráticos, o tempo proporcional de consumo é n ao quadrado. Os de log consomem o tempo proporcional de $\log n$. E os lineares consomem o tempo proporcional de n .

Comentar vantagens e limitações de cada método

As vantagens e limitações vão depender do cenário, dependendo do volume de dados a diferença vai ser quase imperceptível. Mas quando tratamos de um grande volume de dados, usar algoritmos lineares ou de n ao quadrado, pode causar problemas, como demora para mostrar os dados para o usuário. Nesse cenário de grande volume de dados, algoritmos $\log n$ ou $n \log n$ vão ser melhores, mas nos quais tratamos pequeno volume de dados os lineares vão ser também uma boa opção, por as vezes consumir menos memória.

Tendo isso em vista, é importante fazer uma análise antes de escolher qual algoritmo irá usar, não apenas pelo consumo de tempo.

Códigos-fonte das atividades do roteiro

Atividade 1:

```
def bubble_sort(lista):  
    n = len(lista)  
    for i in range(n - 1):
```

```

        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j+1] = lista[j+1], lista[j]
        return lista

lista = [3, 5, 1, 4, 7, 10]

print(bubble_sort(lista))

```

Desafio:

```

import time
from typing import List
import random

def bubble_sort(lista):
    n = len(lista)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            # Verifica se o elemento atual é maior que o próximo
            if lista[j] > lista[j + 1]:
                # TROCA os elementos de lugar
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
    return lista # retorna a lista ordenada

def merge_sort(esquerda, direita):
    resultado = []
    i = j = 0

    # Enquanto houver elementos em ambas as listas
    while i < len(esquerda) and j < len(direita):
        # Compara o menor elemento de cada metade
        if esquerda[i] < direita[j]:
            resultado.append(esquerda[i]) # adiciona o menor

```

```

        i += 1
    else:
        resultado.append(direita[j]) # adiciona o menor da direita
        j += 1

# Adiciona os elementos restantes (de uma das metades)
resultado.extend(esquerda[i:])
resultado.extend(direita[j:])

return resultado #resultado da fusão (merge) ordenada

def quicksort(array):
    if len(array) < 2:
        return array # caso base (vetor pequeno já está ordenado)
    else:
        # Escolhe um pivô aleatoriamente
        indice_pivo = random.randint(0, len(array) - 1)
        pivo = array[indice_pivo]

        # Remove o pivô do array
        restantes = array[:indice_pivo] + array[indice_pivo+1:]

        # Cria lista com valores menores ou iguais ao pivô
        menores = [i for i in restantes if i <= pivo]

        # Cria lista com valores maiores ou iguais ao pivô
        maiores = [i for i in restantes if i >= pivo]

        # Recursão e concatenação final
        return quicksort(menores) + [pivo] + quicksort(maiores)

lista = [3, 5, 1, 9, 7, 10, 6, 8, 53]
tam = 20000

```



```

lst = list(range(tam)) # cria uma lista com 20.000 números
n = len(lst)
meio = n // 2
metade = lst[meio-1]
direita = lst[:metade]
esquerda = lst[metade:]
t0 = time.time()
direita_ordenada = sorted(direita)
esquerda_ordenada = sorted(esquerda)
t1 = time.time()
sorted_time = t1 - t0

# BUBBLE SORT
t0 = time.time()
bubble_res = bubble_sort(lst)
t1 = time.time()
bubble_time = t1 - t0

# MERGE SORT
t0 = time.time()
merge_res = merge_sort(direita_ordenada, esquerda_ordenada)
t1 = time.time()
merge_time = t1 - t0

# QUICKSORT ALEATÓRIO
t0 = time.time()
quick_res = quicksort(lst)
t1 = time.time()
quick_time = t1 - t0

print(f"Bubble Sort tempo: {bubble_time}")
print(f"Merge Sort tempo: {merge_time}")

```

```
print(f"Quik Sort pivô aleatório tempo: {quick_time}")
print(f"Tempo de ordenar a direita e esquerda com sorted: {sorted_time}")
```

Tabela mostrando o tempo de cada um com uma lista de 20.000 elementos:

Bubble Sort Tempo	MergeSort Tempo	QuickSort Pivô Aleatório Tempo	Sorted()
8.558 segundos	0.001 segundos	0.026 segundos	0.0 segundos

Referências

https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/footnotes/tempo.html

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE PESQUISA - BUSCA LINEAR E BUSCA BINÁRIA EM PYTHON

Roteiro 5

Diferença entre pesquisa exaustiva e pesquisa por divisão

Pesquisa exaustiva o algoritmo gera todas as possibilidades possíveis para o problema, até encontrar a melhor solução para ele. A divisão por conquista, o problema é dividido em partes menores, cada parte menor também é aplicado o algoritmo.

Custos de busca linear e binária em termos de complexidade e de requisitos de ordenação

Busca linear é $O(n)$ e a busca binária é $O(\log n)$, a pesquisa binária apesar de ser mais rápida, apenas funciona em elementos já ordenados.

Códigos-fonte das atividades do roteiro

Atividade 1:

```
# pesquisa linear
def linear_search(seq, alvo):
    # é o index e v é o valor
    for i, v in enumerate(seq):
        # se o valor for igual o alvo, retornará o index onde o item se encontra
        if v == alvo:
            return i
    # se não encontrar nada retorna -1
    return -1

def binary_search(seq, alvo):
    # ini recebi 0, fim recebi a largura menos -1
    ini, fim = 0, len(seq) - 1

    while ini <= fim:
        # pega o meio do array
```

```

    meio = (ini + fim) // 2
    # se encontrar o item, retorna a posição dele
    if seq[meio] == alvo:
        return meio

    if seq[meio] < alvo:
        ini = meio + 1
    else:
        fim = meio - 1

    return -1

lista = [1, 2, 3, 4, 5, 6, 7]

print(linear_seach(lista, 7))
print(binary_search(lista, 3))

```

Desafio:

```

import random, timeit
def pesquisa_linear(lista, alvo):

    #Realiza uma busca linear no array. Percorrendo cada elemento até
    encontrar o alvo, se não achar retorna -1

    for i in range(0, len(lista)):
        if lista[i] == alvo:
            return i
    return -1

def pesquisa_binaria(lista, alvo):
    """
    Realiza uma busca binária na lista.
    Requer que a lista esteja ordenada.

```

```

Divide o espaço de busca pela metade a cada iteração.
"""
ini = 0
fim = len(lista) - 1

while ini <= fim:
    meio = (ini + fim) // 2

    if lista[meio] == alvo:
        return meio
    if lista[meio] < alvo:
        ini = meio + 1
    else:
        fim = meio - 1

return -1

# teste com 250 elementos
n = 250
# Vair gerar uma lista com 250 números entre 0 e 100.000
lista = [random.randint(0, 100_000) for _ in range(n)]
alvo = 30 # Define o número que será procurado

# Mede o tempo da busca linear
t_lin = timeit.timeit(lambda: pesquisa_linear(lista, alvo), number=1)
# Ordena a lista para a busca binária funcionar corretamente
ordenada = sorted(lista)
# Mede o tempo da busca binária, o number=1 significa que só vai executar
uma vez
t_bin = timeit.timeit(lambda: pesquisa_binaria(ordenada, alvo), number=1)

print(f"Busca linear com 250 elementos: {t_lin:.6f}s")
print(f"Busca binária com 250 elementos: {t_bin:.6f}s")

```

```

# teste com 500 elementos
n = 500
# Gera uma nova lista com 500 elementos
lista = [random.randint(0, 100_000) for _ in range(n)]
alvo = 30

t_lin = timeit.timeit(lambda: pesquisa_linear(lista, alvo), number=1)
ordenada = sorted(lista)
t_bin = timeit.timeit(lambda: pesquisa_binaria(ordenada, alvo), number=1)

print(f"Busca linear com 500 elementos: {t_lin:.6f}s")
print(f"Busca binária com 500 elementos: {t_bin:.6f}s")

# teste 1000 elementos
n = 1000
# Gera uma nova lista com 1000 elementos
lista = [random.randint(0, 100_000) for _ in range(n)]
alvo = 30

t_lin = timeit.timeit(lambda: pesquisa_linear(lista, alvo), number=1)
ordenada = sorted(lista)
t_bin = timeit.timeit(lambda: pesquisa_binaria(ordenada, alvo), number=1)

print(f"Busca linear com 1000 elementos: {t_lin:.6f}s")
print(f"Busca binária com 1000 elementos: {t_bin:.6f}s")

```

Quantidade de elementos na lista	Busca linear tempo	Busca binária tempo
250 elementos	0.000010s	0.000004s
500 elementos	0.000026s	0.000004s
1000 elementos	0.000029s	0.000003s

Referências

<https://prezi.com/p/qna8aulg7qnu/pesquisa-exaustiva/>

https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/divide-and-conquer.html

<https://www.freecodecamp.org/portuguese/news/uma-introducao-a-complexidade-temporal-dos-algoritmos/>

<https://www.hackerearth.com/practice/algorithms/searching/binary-search/tutorial/>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): TABELAS DE DISPERSÃO (HASH TABLES) E OS HEAPS EM PYTHON

Relatório 6

Tabelas de dispersão, explicar colisões e tratamentos

Uma tabela de dispersão (Hash Table) é uma maneira de organizar tabelas de símbolos, que armazena objetos de algum tipo. Tabelas de símbolos são uma generalização da ideia de endereçamento direto. Cada item dentro dessa tabela possui uma chave, associando as chaves aos valores, permitindo buscas rápidas e eficientes. Colisão é o nome que se dá quando duas chaves são indicadas para o mesmo espaço, um dos métodos de tratamentos se chama separate chaining, que consiste em que em cada índice h da tabela haverá uma lista encadeada que armazena todos os itens dessa chave.

heaps binários e justificar eficiência em filas de prioridade

Um heap binário que representa uma árvore binária completa, nela todos os níveis estão completos, exceto o último. Existem dois tipos de heaps binários: o max heap e min heap. Max heap os pais têm valores maiores ou iguais ao dos filhos, O min heap os pais têm valores menores ou iguais ao dos filhos.

As filas de prioridades são estruturas de dados, onde os elementos são organizados de uma maneira que tenham uma determinada prioridade, que vai ser definida por critérios. São eficientes por facilitarem na busca, inserção de dados, além de sua flexibilidade, sendo versátil para diversos cenários.

Códigos-fonte das atividades do roteiro

Atividade 1:

```
import heapq
tarefa = []

heapq.heappush(tarefa, (3, "Enviar relatório."))
heapq.heappush(tarefa, (1, "Responder e-mails"))
heapq.heappush(tarefa, (2, "Revisar código"))
```



```
while tarefa:
    prioridade, nome = heapq.heappop(tarefa)
    print(prioridade, nome)
```

Desafio:

```
import heapq
import random
import timeit

n = 50_000
# Cria uma lista com 50 mil números aleatórios entre 1 e 1 milhão
dados = [random.randint(1, 1_000_000) for _ in range(n)]

def heap_invertendo_sinal():
    heap = []
    # heappush insere elemento mantendo a propriedade de heap
    # Aqui invertemos o sinal (-v) para simular uma max-heap,
    # pois o heapq por padrão é um min-heap
    for v in dados:
        heapq.heappush(heap, -v)
    # heappop remove o menor elemento (que, com sinal invertido,
    # equivale ao maior valor original)
    while heap:
        heapq.heappop(heap)

class MaxValue:
    __slots__ = ("valor",) # economiza memória, define atributos fixos
    def __init__(self, valor):
        self.valor = valor

    # Método de comparação especial usado pelo heapq
    # Aqui invertemos o critério: maior valor é considerado "menor"
```

```

# para o heapq, simulando uma max-heap real.
def __lt__(self, outro):
    return self.valor > outro.valor

def __repr__(self):
    return f"{self.valor}"

def heap_classe_propria():
    heap = []
    # heappush insere objetos da classe MaxValue
    for v in dados:
        heapq.heappush(heap, MaxValue(v))
    # heappop remove mantendo a ordem definida em __lt__
    while heap:
        heapq.heappop(heap)

# Mede o tempo de execução de cada abordagem apenas 1 vez
tempo_sinal = timeit.timeit(heap_invertendo_sinal, number=1)
tempo_classe = timeit.timeit(heap_classe_propria, number=1)

# Mostra os resultados de forma formatada
print(f"Invertendo sinal: {tempo_sinal:.4f} s")
print(f"Classe própria: {tempo_classe:.4f} s")

```

Tabela de resultados:

Invertendo sinal	Classe própria
0,0160s	0,0670s

Referências

<https://www.ime.usp.br/~pf/mac0122-2002/aulas/hashing.html>

<https://www.dio.me/articles/estrutura-de-dados-tabela-hash>

entendendo algoritmos, Aditya Y.Bhargava, página: 106

<https://elemarjr.com/clube-de-estudos/artigos/binary-heap-e-heapsort-o-que-sao-e-com-o-funcionam/>

<https://www.estrategiaconcursos.com.br/blog/filas-prioridades-heap-cnu-ti/>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE GRAFOS - DIJKSTRA, BELLMAN-FORD, KRUSKAL E PRIM EM PYTHON

Roteiro 7

Diferenças entre caminhos mínimos de fonte única e árvores geradoras mínimas.

Problemas de caminhos mínimos é um algoritmo para se calcular o caminho mínimo para cada vértice de um grafo, exemplos: Bfs (busca em largura) e o Dijkstra. Árvores geradoras mínimas em grafos ponderados, selecionam um subconjunto de arestas com custo total mínimo sem acabar violando a conectividade global do conjunto de vértices.

Apontar condições de aplicabilidade (pesos negativos, denso × esparso)

Para grafos de pesos negativos, algoritmos como Dijkstra, não funcionam, precisa-se usar algoritmos como Bellman-Ford.

A densidade de um grafo é uma função entre o número de vértices e o de arestas, o grafo denso é um que possui uma densidade alta, próxima de 1, o grafo esparso é o que possui uma baixa, próxima de 0.

Códigos-fonte das atividades do roteiro

Atividade 1:

```
import heapq
def dijkstra(grafo, origem):
    dist = {v: float('inf') for v in grafo}
    dist[origem] = 0
    fila = [(0, origem)]

    while fila:
        d, u = heapq.heappop(fila)
        if d > dist[u]:
            continue
        for v, w in grafo[u]: # percorre vizinhos de u
            novo = d + w
            if novo < dist[v]:
                dist[v] = novo
                heapq.heappush(fila, (novo, v))
    return dist
```

```

# Grafo como dicionário
grafo = {
    "A": [],
    "B": [],
    "C": []
}

def adicionar_aresta_lista(u, v, peso):
    grafo[u].append((v, peso))
    grafo[v].append((u, peso)) # não direcionado

# Adicionando arestas
adicionar_aresta_lista("A", "B", 5)
adicionar_aresta_lista("B", "C", 2)

print("\nLista de Adjacência (com pesos):")
for vertice, vizinhos in grafo.items():
    print(f"{vertice}: {vizinhos}")

print("\nDijkstra a partir de B:")
print(dijkstra(grafo, "B"))

```

Desafio

```

import random
import timeit
import heapq

# Algoritmo de Dijkstra para encontrar caminhos mínimos em grafos sem
# pesos negativos
def dijkstra(grafo, origem):
    # Inicializa todas as distâncias como infinito
    dist = {v: float('inf') for v in grafo}

```

```

# Distância da origem para ela mesma é 0
dist[origem] = 0
# Fila de prioridade (heap) com (distância, vértice)
fila = [(0, origem)]

while fila:
    # Remove o vértice com menor distância da fila
    d, u = heapq.heappop(fila)
    # Se encontramos um caminho melhor anteriormente, ignora
    if d > dist[u]:
        continue
    # Para cada vizinho do vértice atual
    for v, w in grafo[u]:
        # Calcula nova distância
        novo = d + w
        # Se encontrou caminho melhor, atualiza
        if novo < dist[v]:
            dist[v] = novo
            heapq.heappush(fila, (novo, v))
return dist

# Algoritmo de Bellman-Ford para caminhos mínimos (funciona com pesos negativos)
def bellman_ford(grafo, origem):
    # Coleta todos os vértices do grafo
    vertices = set(grafo.keys())
    for arestas in grafo.values():
        for destino, _ in arestas:
            vertices.add(destino)

    # Inicializa distâncias e antecessores
    dist = {v: float('inf') for v in vertices}
    antecessor = {}
    dist[origem] = 0

    # Relaxamento das arestas |V|-1 vezes
    for _ in range(len(vertices) - 1):

```

```

        for u in vertices:
            for v, peso in grafo.get(u, []):
                if dist[u] + peso < dist[v]:
                    dist[v] = dist[u] + peso
                    antecessor[v] = u

# Verifica se há ciclos de peso negativo
for u in vertices:
    for v, peso in grafo.get(u, []):
        if dist[u] + peso < dist[v]:
            raise ValueError("Ciclo de peso negativo detectado")

return dist, antecessor

# Geração de grafo aleatório para teste
num_vertices = 200
num_arestas = 400
vertices = [i for i in range(num_vertices)]

# Cria grafo vazio
grafo = {v: [] for v in vertices}

# Adiciona arestas aleatórias
for _ in range(num_arestas):
    u = random.choice(vertices)
    v = random.choice(vertices)
    if u != v:
        peso = random.randint(1, 20)
        grafo[u].append((v, peso))

# Testa os algoritmos de caminho mínimo
origem = int(input("Origem: "))

# Mede tempo de execução
tm_dij = timeit.timeit(lambda: dijkstra(grafo, origem), number=1)
tm_bf = timeit.timeit(lambda: bellman_ford(grafo, origem), number=1)

```

```

print(f"Dijkstra: {tm_dij:.4f}s")
print(f"Bellman-Ford: {tm_bf:.4f}s")

## Comparação de algoritmos para Árvore Geradora Mínima

# Cria novo grafo não-direcionado para MST
vertices = 200
arestas = 400
vertice = [i for i in range(vertices)]

# Estruturas para Kruskal e Prim
aresta = []
grafo_adj = {v: [] for v in vertice}

# Gera arestas bidirecionais
for _ in range(arestas):
    u, v = random.sample(vertice, 2)
    peso = random.randint(1, 20)
    aresta.append((peso, u, v))
    grafo_adj[u].append((v, peso))
    grafo_adj[v].append((u, peso))

# Algoritmo de Kruskal para MST (usa Union-Find)

# Estruturas para Union-Find
parent = {}
rank = {}

# Find com compressão de caminho
def find(u):
    if parent[u] != u:
        parent[u] = find(parent[u])
    return parent[u]

# Union por rank
def union(u, v):
    u_root = find(u)

```



```

v_root = find(v)
if u_root == v_root:
    return False # Já estão no mesmo conjunto
# Une pela altura (rank)
if rank[u_root] < rank[v_root]:
    parent[u_root] = v_root
else:
    parent[v_root] = u_root
    if rank[u_root] == rank[v_root]:
        rank[u_root] += 1
return True

def kruskal(vertice, aresta):
    # Inicializa estruturas Union-Find
    for v in vertice:
        parent[v] = v
        rank[v] = 0
    mst = [] # Árvore geradora mínima
    total = 0 # Peso total

    # Ordena arestas por peso e processa da menor para maior
    for peso, u, v in sorted(aresta):
        if union(u, v): # Se não forma ciclo
            mst.append((u, v, peso))
            total += peso
    return mst, total

# Algoritmo de Prim para MST (usa abordagem gulosa com heap)
def prim(vertice, grafo_adj, inicio):
    visitado = set([inicio]) # Vértices já na MST
    heap = [] # Heap de arestas candidatas

    # Adiciona arestas do vértice inicial
    for v, peso in grafo_adj[inicio]:
        heapq.heappush(heap, (peso, inicio, v))

    mst = []

```

```

total = 0

# Expande a MST até incluir todos os vértices
while heap and len(visitado) < len(vertice):
    peso, u, v = heapq.heappop(heap)
    if v not in visitado:
        visitado.add(v)
        mst.append((u, v, peso))
        total += peso
        # Adiciona novas arestas candidatas
        for to, w in grafo_adj[v]:
            if to not in visitado:
                heapq.heappush(heap, (w, v, to))
    return mst, total

# Comparação de desempenho entre Kruskal e Prim
inicio_kruskal = timeit.default_timer()
mst_kruskal, total_kruskal = kruskal(vertice, aresta)
tempo_kruskal = timeit.default_timer() - inicio_kruskal

inicio_prim = timeit.default_timer()
mst_prim, total_prim = prim(vertice, grafo_adj, vertice[0])
tempo_prim = timeit.default_timer() - inicio_prim

print(f"Kruskal: peso total = {total_kruskal}, tempo = {tempo_kruskal:.6f}s")
print(f"Prim:    peso total = {total_prim}, tempo = {tempo_prim:.6f}s")

```

Tabela de tempo de execução:

Dijkstra	Bellman-Ford	Kruskal	Prim
0,0001s	0,0063s	0,000214s	0,000181s

Referências

https://hokama.com.br/disciplinas/cic111_2022s1/aula22-handout.pdf

Livro texto, unip. Algoritmos e estruturas de dados em Python, autor: Prof. Tarcísio de Souza Peres, página 277

https://professor.ufop.br/sites/default/files/george/files/a04_rerepresentacao_de_grafos_0.pdf

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): TÉCNICAS DE DIVISÃO E CONQUISTA E DE PROGRAMAÇÃO DINÂMICA EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório (2 a 3 páginas) contendo:

Resumo Teórico:

- A explanação das diferenças estruturais entre divisão e conquista e programação dinâmica.
- A justificativa dos ganhos obtidos com memoização ou tabulação nos problemas escolhidos;

Descrição das Classes Criadas:

- A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
- A apresentação dos tempos medidos em tabela.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resumo Teórico	2,0	A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
Organização e Comentários do Código	3,0	A apresentação dos tempos medidos em tabela.
Funcionalidade do Código	3,0	A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
Criatividade e Aprimoramentos	2,0	Análise gráfica, discussão sobre consumo de memória ou casos extremos..

